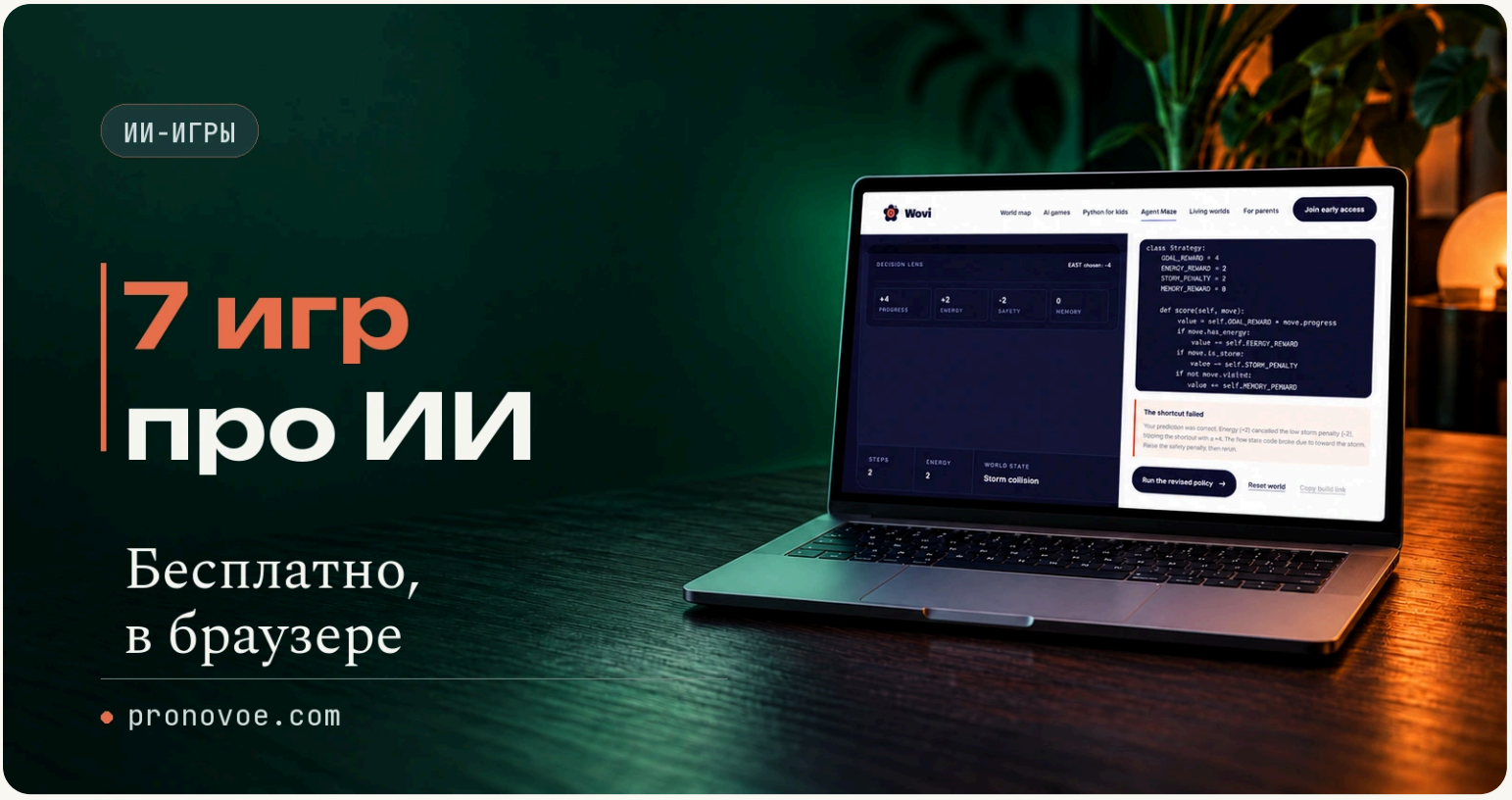


7 бесплатных игр про ИИ: что попробовать и чему они научат

7 бесплатных игр про ИИ: что попробовать и чему они научат

Начальный уровень

Если хочется начать прямо сейчас, открой Agent Maze, выбери прогноз SHORTCUT и запусти робота со штрафом Low -2. Он возьмёт короткий путь и попадёт в бурю. Затем переключи Safety на High -12 и повтори запуск. На экране появится безопасный маршрут и надпись «Beacon reached». За пять минут становится видно главное: агент выбирает шаг по заданной математической оценке

Ниже ещё шесть опытов. У каждого есть ссылка, сложность и условие бесплатного запуска. Часть проектов работает как прототип, Tensor Trust требует аккаунт, а Steerscope пока показывает предварительную версию (preview)

▶ Открыть содержание

ПОДЕЛИТЬСЯ Telegram WhatsApp VK

Agent Maze: когда цель без ограничения ведёт в бурю

В Agent Maze сразу видно, как одно число меняет маршрут робота. Это простая игра для современного браузера с JavaScript, без аккаунта

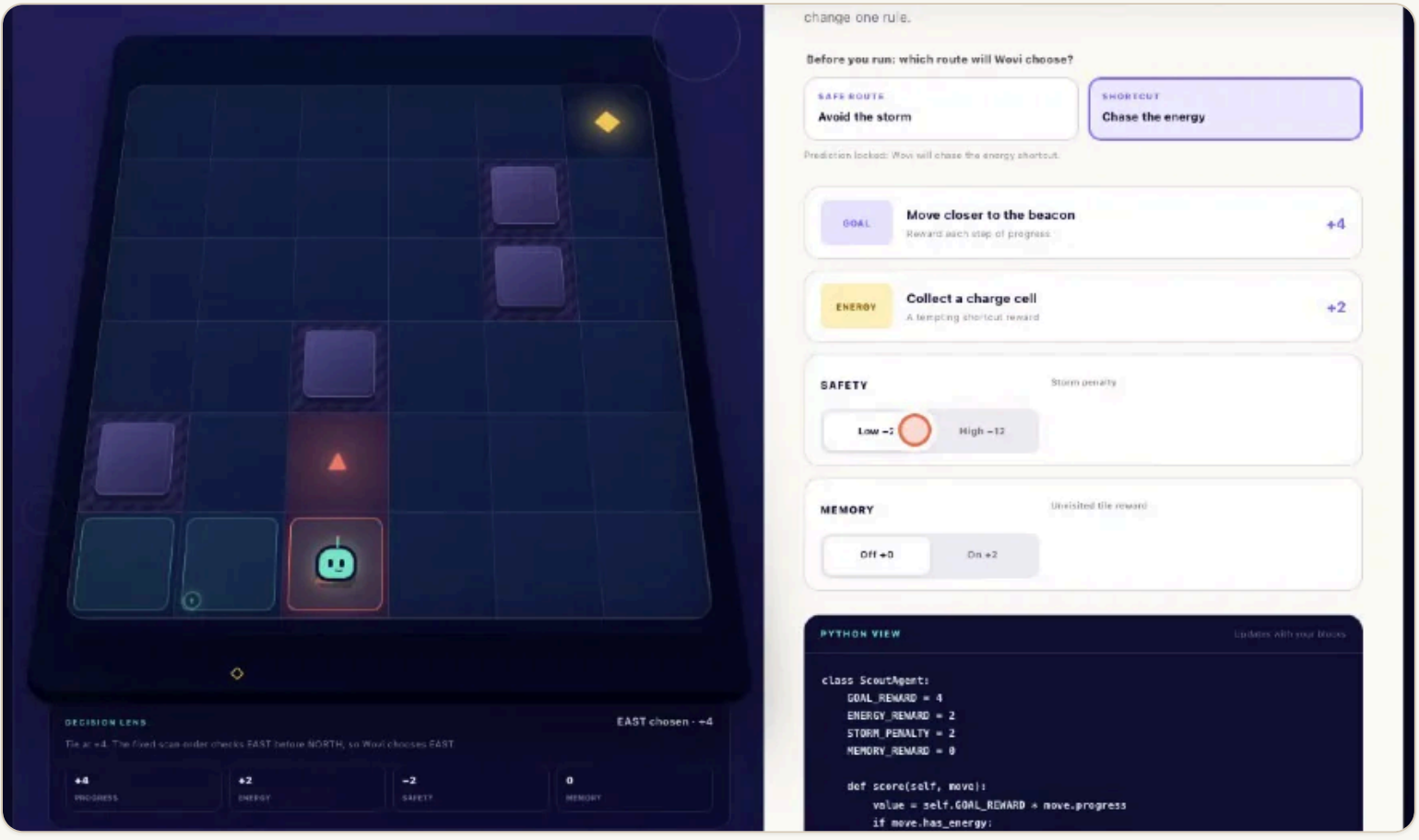
Игра объясняет награды, штрафы и конфликт короткого пути с безопасностью. Робот не понимает бурю как человек. Он складывает числа, поставленные разработчиком

Начни с прогноза SHORTCUT и варианта Low -2. Затем переключи Safety на High -12 и повтори запуск

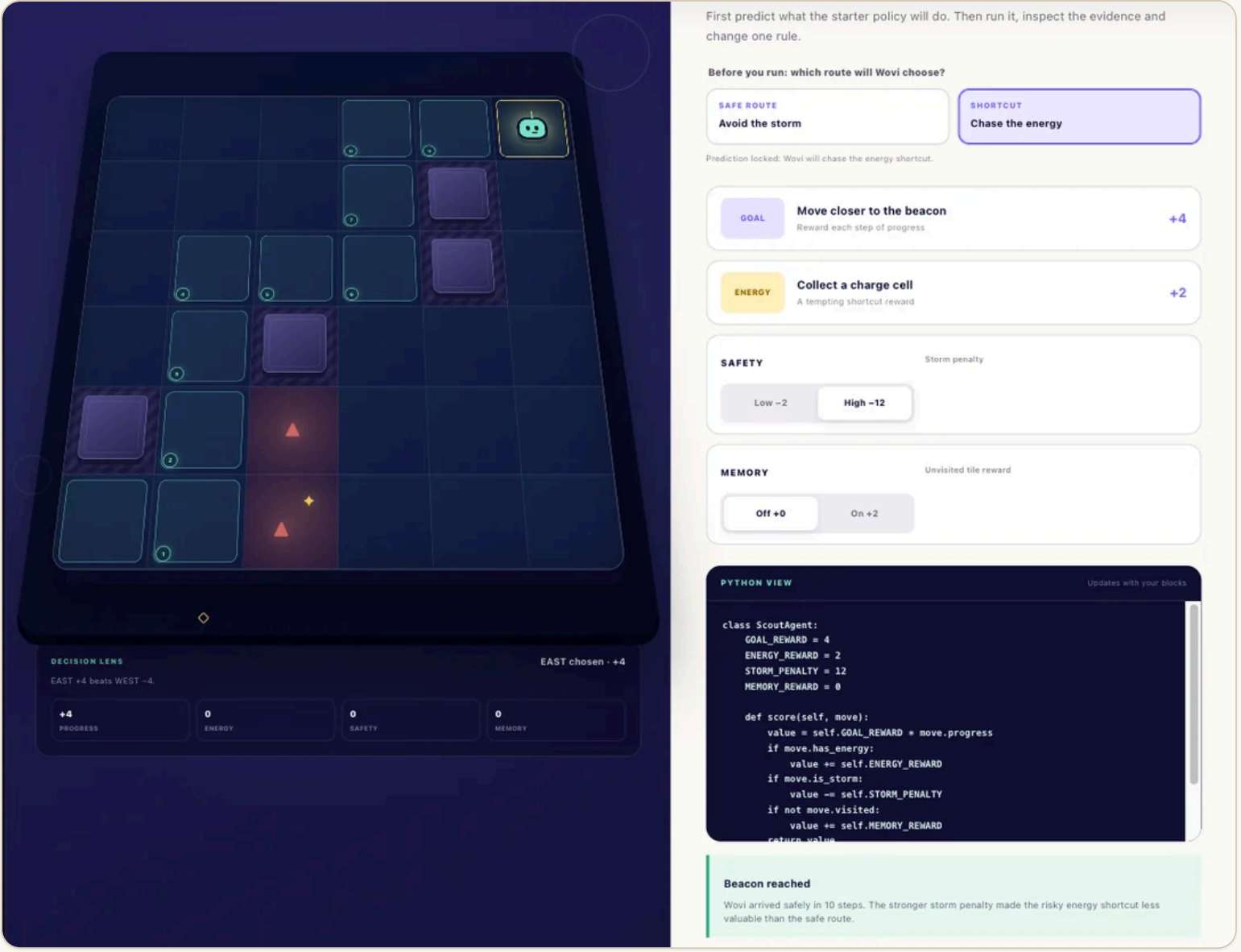
Сейчас доступен публичный игровой прототип: на странице написано «No account · 5-minute mission». Аккаунт и оплата для этой миссии не нужны

> **Результат: при -2 робот идёт через бурю, при -12 доходит до маяка за 10 шагов.** При -2 опасный и безопасный варианты получают по +4, и робот выбирает первый путь через бурю

▶ Это обучение ChatGPT?



- При небольшой потере путь через бурю остаётся выгодным по формуле, хотя миссия останавливается



- После увеличения потери до 12 баллов безопасный обход выигрывает, на экране появляется Beacon reached

PlayPlain: почему в strawberry ИИ видит две `r`

Открой BPE Tokenization в PlayPlain в браузере. Это простой опыт с английским словом, вход в аккаунт не требуется

Он показывает, что модель получает слово частями, которые называют токенами. Буквы внутри крупного фрагмента учебная модель не пересчитывает по отдельности

Оставь `strawberry` и букву `r` : сначала увидишь «token-level guess: 2 · truth: 3». Затем нажми `spell it out first` . Слово разложится по буквам, и ответ станет 3

> **Результат: «token-level guess: 2» сменился на правильный ответ 3.** Каждая буква стала отдельным элементом

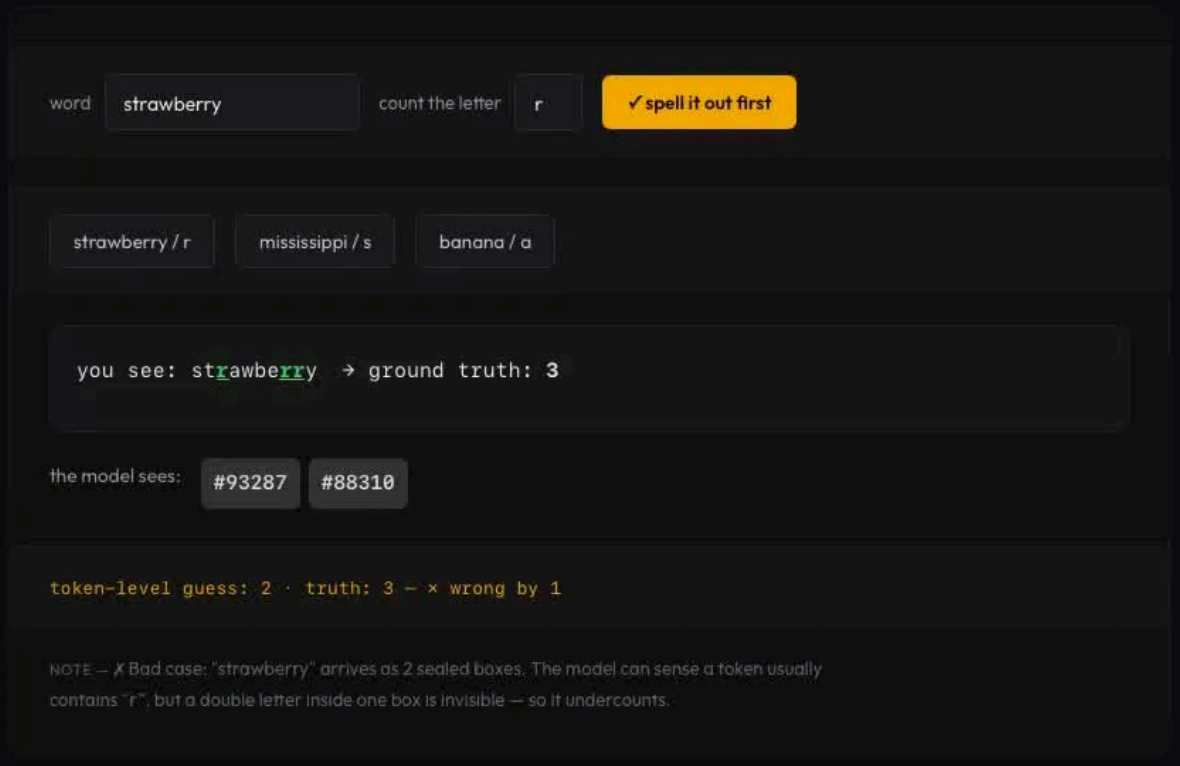
Интерактив сейчас запускается без входа и оплаты. Кнопка Sign in есть, но для опыта со strawberry она не понадобилась

▶ **Это настоящий токенизатор ChatGPT?**

EP 03

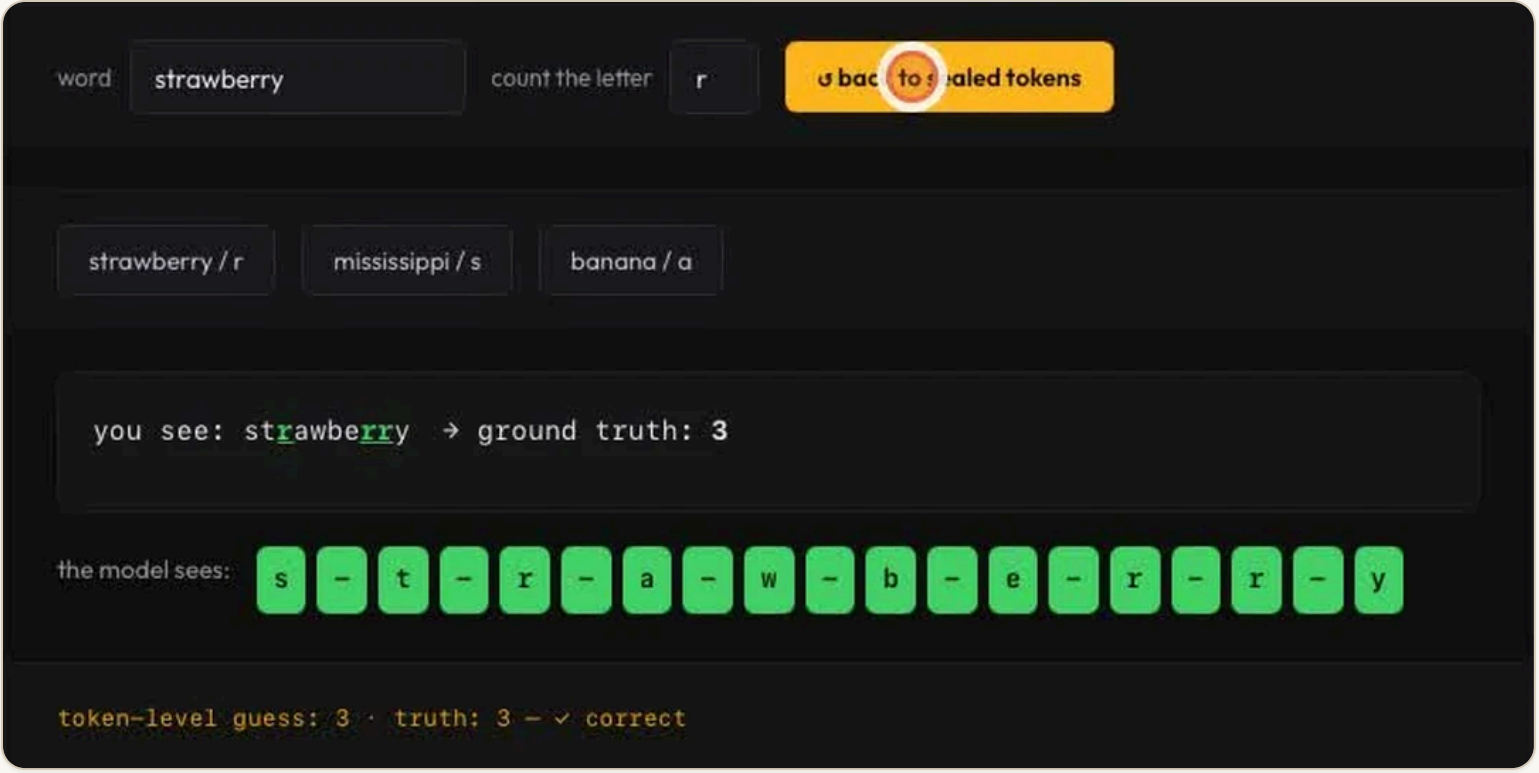
The strawberry problem — letters are invisible inside tokens

Here's the famous one. You see **letters**; the model sees **opaque token IDs**. "strawberry" arrives as two sealed boxes — the model never receives an r, it receives #24807 and #19772. Ask it to count letters and it can only guess from what tokens *usually* contain. Try it, then hit "spell it out" to fix it the way prompt engineers actually do.



> **The fix is the proof:** hyphenate the word and every letter becomes its own token — suddenly the count is trivial. That's why "think step by step, spell the word first" works on real models. The model isn't bad at counting; it's counting things it literally cannot see.

- Пока слово лежит в двух крупных кусках, учебная модель отвечает 2 при правильном ответе 3



- После команды spell it out first каждая буква становится отдельным элементом, и ответ меняется на 3

Art Palette: как ИИ ищет картины по пяти цветам

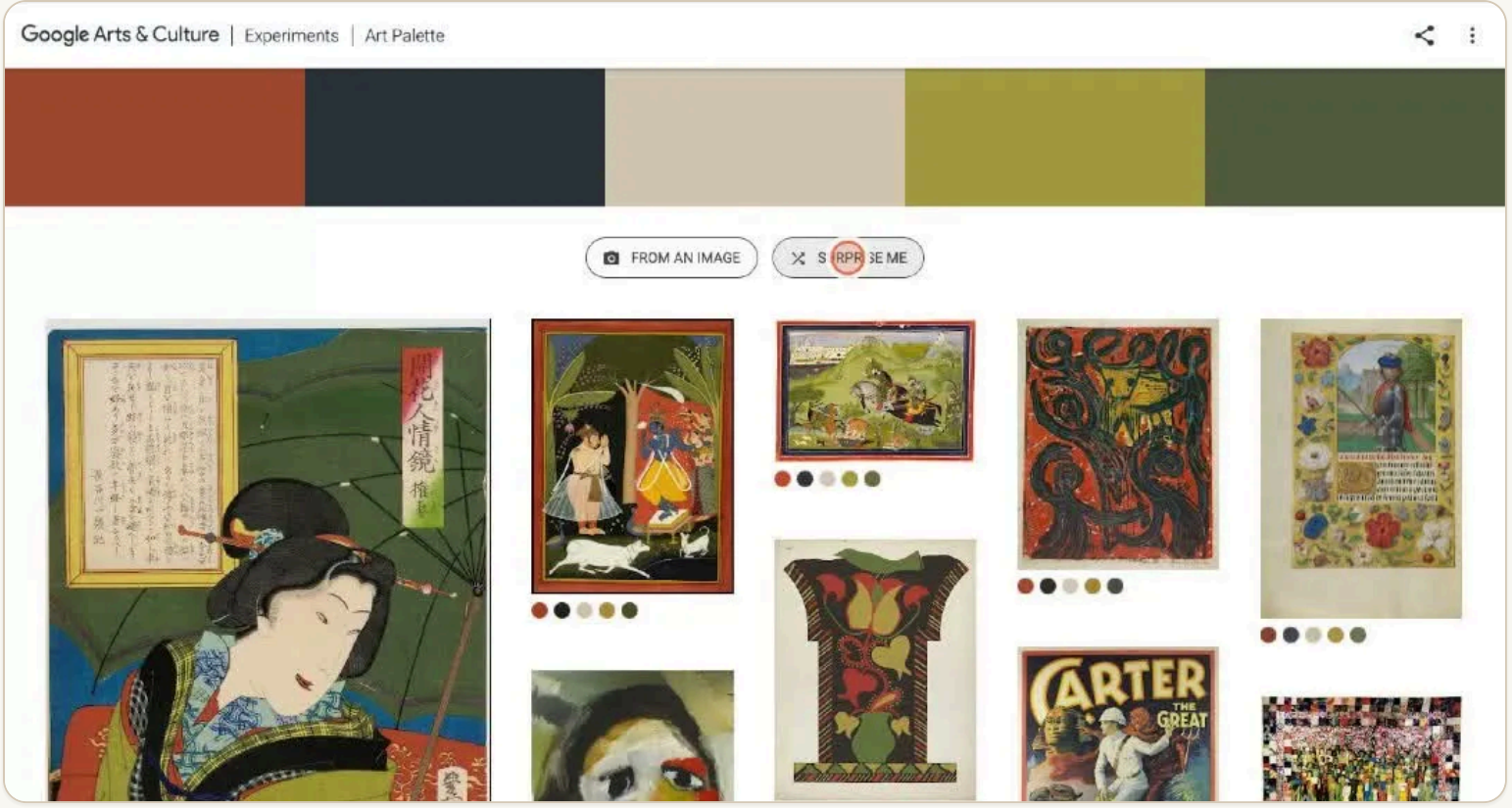
В Art Palette можно увидеть результат сразу: выбираешь пять цветов, а ниже меняется подборка картин. Сложность простая

Так работает поиск похожих объектов по признакам. Здесь признак наглядный: пять цветов, а не сюжет картины

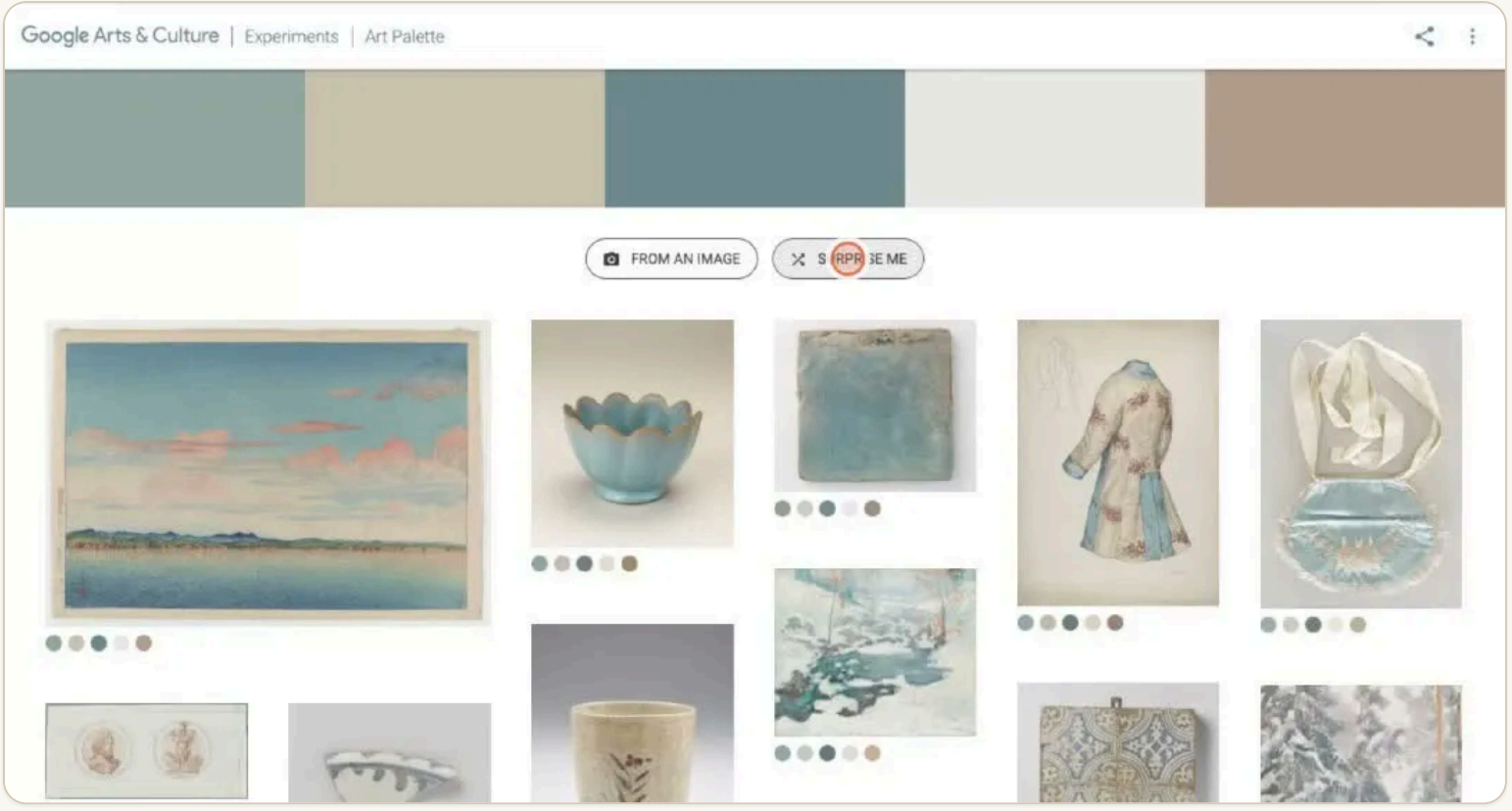
Для запуска достаточно браузера и кнопки Surprise Me. Её можно нажать вместо ручного выбора цветов или загрузки изображения: пять цветов наверху сменятся, а ниже появится другой набор работ

▶ **Что уходит в Google, если загрузить своё фото**

> **Результат: пять цветов наверху сменились, ниже другой набор картин.** Меняется выбранный набор признаков, и алгоритм перестраивает выдачу



- Первый набор из пяти цветов даёт работы с похожей тёплой палитрой



- После Surprise Me меняются пять признаков-цветов и вся выдача изображений

AlphaLudo: нейросеть делает ход и показывает свои шансы

В AlphaLudo нейросеть оценивает допустимые ходы и вероятность победы в лудо, где часть событий зависит от кубика. Если правила лудо незнакомы, сложность средняя

Нужен браузер с JavaScript. Первая загрузка занимает около 6 МБ. Для объёмной доски нужен WebGL, но предусмотрен плоский режим

Начни с кубика: нажми его, сделай доступный ход и следи за строкой «Who's winning». Она меняется после ходов и показывает оценку самой сети

> **Результат: после твоего хода строка «Who's winning» показывает новые шансы.**

По описанию автора, аккаунт, сервер и оплата не нужны. Игра работает на устройстве, не отправляет ходы на сервер и не использует аналитику, cookies или tracking

Твоя партия не дообучает сеть. В браузере работает уже обученная модель, поэтому одинаковая позиция даёт одинаковый выбор при полной силе

Agent Breaker: учебные атаки на ИИ-агентов

В Agent Breaker агент читает документы и описания инструментов. Игра показывает prompt injection: текст, который выглядит как данные, агент может принять за команду

Это опыт средней или высокой сложности. Понадобятся браузер, английский интерфейс и внимательное чтение цели уровня

Начни с Curs-ed CodeReview, открой Level 1 Novice и прочитай Attack Objective. Цель написана слева, результат атаки появляется в макете приложения справа

> **Результат: в макете приложения справа видно, чем закончилась атака.**

Каталог и первый уровень сейчас открываются без аккаунта и оплаты

Работай только внутри учебной площадки. То, что разрешено целью игры, не даёт разрешения атаковать чужие сайты, ботов или аккаунты

Tensor Trust: защита промпта против атаки

В Tensor Trust один игрок пишет защитное правило и вымышленный пароль, а другой пытается заставить модель сказать «access granted» без правильного пароля. Сложность средняя

Для запуска нужен аккаунт и согласие с условиями исследовательского проекта. На публичной странице нет цены и шага оплаты, но при создании аккаунта пользователь соглашается на использование данных в исследовании

Первый шаг - прочитать Consent and Terms. Затем используй только вымышленные данные, создай простое защитное правило и посмотри, как другие игроки пытаются его обойти

> Тексты, отправленные в игру, планируют публиковать для исследования вместе со временем отправки и случайными идентификаторами. Реальные пароли, имена и другую

личную информацию туда вводить нельзя

Steerscope: посмотреть прогноз по слоям и изменить направление

На Steerscope показаны прогнозы следующего токена от нижних слоёв к верхним и влияние управления направлением активаций (steering) на продолжение. Это самый сложный опыт в списке

Для запуска нужен современный браузер. Для полноценного локального вывода автор указывает WebGPU и совместимый GPU, но текущая предварительная версия открывается без аккаунта и оплаты. Автор заявляет локальные вычисления без платы за посещение

Начни с готовой фразы: нажми токен в продолжении, посмотри лестницу из 25 слоёв и подвигай один ползунок, например Formality

▶ **Steerscope показывает настоящую модель?**

Как выбрать первую игру

Для первого захода бери одну из трёх простых игр. В Agent Maze меняется решение робота, в PlayPlain видна ошибка подсчёта букв, а в Art Palette результат меняется вместе с цветами

ИГРА	ЧТО ПОНЯТЬ	ПЕРВЫЙ ШАГ	СЛОЖНОСТЬ	НУЖЕН ЛИ АККАУНТ
Agent Maze	Почему агент выбирает плохой путь	Сравни -2 и -12	Простая	Нет
PlayPlain	Почему ИИ ошибается в подсчёте букв	Нажми spell it out first	Простая	Нет
Art Palette	Как искать картины по цветам	Нажми Surprise Me	Простая	Нет
AlphaLudo	Как сеть оценивает шансы на победу	Брось кубик	Средняя	Нет
Agent Breaker	Как инструкция в данных влияет на агента	Открой первый уровень	Средняя	Для первого уровня нет
Tensor Trust	Как обходят защитное правило	Прочитай условия	Средняя	Да

ИГРА	ЧТО ПОНЯТЬ	ПЕРВЫЙ ШАГ	СЛОЖНОСТЬ	НУЖЕН ЛИ АККАУНТ
Steerscope	Как меняется прогноз по слоям	Нажми токен	Высокая	Нет

Что открыть дальше на сайте

Пять материалов ниже помогут перейти от игровых примеров к применению ИИ в собственных задачах

- Как устроены ИИ-агенты и где они ошибаются
- Как Claude и Codex работают в VS Code
- Как выбрать модель и уровень усилия
- Как пользоваться веб-поиском в Claude
- Как объяснить ИИ свой контекст

7 мин чтения На главную

Источники

- Agent Maze: живая игра и видимая формула
- PlayPlain: BPE Tokenization
- Google Arts & Culture: Art Palette
- Google Arts & Culture: как работает Art Palette
- AlphaLudo: устройство игры и приватность
- AlphaLudo: исследовательский журнал
- Lakera: Agent Breaker
- Tensor Trust: игра и предупреждение о данных
- Tensor Trust: Consent and Terms
- Steerscope: живая страница
- Steerscope: исходный код и честный статус

Памятка перед первым запуском

Выбери один опыт, а после игры сравни наблюдение с объяснением в статье

СОХРАНИ СЕБЕ

Скопировать список

- [ ] Открой Agent Maze без регистрации и сравни Low -2 с High -12
- [ ] Если интереснее работа со словами, проверь ответ 2 и ответ 3 в PlayPlain
- [ ] Если нужен быстрый визуальный результат, смени пять цветов в Art Palette

Частые вопросы

Это действительно игры, а не обычные статьи?

У всех семи есть действие и наблюдаемый ответ. Но формат разный: Agent Maze и AlphaLudo ближе к обычной игре, PlayPlain и Art Palette являются интерактивными экспериментами, Steerscope сейчас работает как предварительная версия

Они работают на русском языке?

Нет. Интерфейсы в основном английские. Для первых трёх достаточно нескольких кнопок, а Agent Breaker, Tensor Trust и Steerscope требуют читать объяснения внимательнее

Можно ли вводить реальные данные?

В Tensor Trust нельзя вводить то, что не должно стать публичным. В остальных играх тоже лучше использовать вымышленные примеры. Art Palette отдельно описывает обработку палитры загруженного изображения

Какая игра самая простая для первого раза?

Agent Maze, если хочется понять решение агента, или Art Palette, если хочется мгновенный визуальный результат без терминов

ПОДЕЛИТЬСЯ Telegram WhatsApp VK

Непонятно, что на экране? Спроси Claude. Попроси объяснить по одному шагу.

Ты

Открываю одну из игр про ИИ: Agent Maze, PlayPlain или Art Palette. Сначала спроси, какую игру я открыл и что вижу на экране. Объясняй простыми словами по одному шагу и после каждого жди моего ответа. Не проси вводить пароли и личные данные.

Claude

Начнём с экрана, который у тебя открыт. Напиши название игры и опиши, что видишь.

Скопировать

Что можно делать с нейросетями, если не знаешь кода?

Чтобы почувствовать размах, посмотри Концентрат Ивана Сергеева. ИИ-агенты на твоих глазах собирают продукт, контент, сайты и воронки без кода. Это основа под любую цель: свои услуги, партнёрка, чужой бизнес. Три дня бесплатно, VIP с четвёртым днём - 990 ₽

Забрать Концентрат

Уже знаешь, чего хочешь? Тогда в мастер-группу Ивана: система продаж на ИИ без продюсера и технаря, сопровождение и сообщество

Заполнить анкету предзаписи

Набор открывается редко, кто записался заранее, получает лучшую цену и бонусы. Платить сейчас не нужно

Реклама. ИП Сергеев И. С., ИНН 352511695540. erid: 2Vtzqvk2p6q

СЛЕДУЮЩИЙ ШАГ

Команда ИИ-агентов для блога

Посмотреть, как агенты работают вместе

Политика данных

ГОТОВЫЕ ПРОМПТЫ



15 готовых промптов: от требований и плана до тестов, git и скилла

15 задач для Claude Code и Codex: готовые промпты

Пошаговая инструкция

Если начать с команды «сделай приложение», агент быстро напишет код, но тебе придётся угадывать, что он решил за тебя. Здесь работа устроена наоборот: сначала записываем результат, затем шаг за шагом уменьшаем число догадок. Так легче остановить ошибку до того, как она разойдётся по проекту

Это 15 готовых промптов для Claude Code и Codex. Во всех уже указан один пример: локальное веб-приложение «Список дел» с тестовой базой SQLite, то есть отдельным файлом с учебными задачами. Эти промпты проводят учебный проект от требований до сквозной проверки и помогают сохранить инструкцию для повторной приёмки

ПОДГОТОВЛЕНО 16 ОКТЯБРЯ 2026

▶ Открыть содержание

Как пользоваться: создай отдельную папку «Список дел - учебная копия» и открой её в Claude Code или Codex. Если тренируешься на существующем проекте, сначала сделай его отдельную копию и работай только в ней, не в рабочей папке. Нажми «Скопировать», вставь промпт в поле агента и отправь. Переходи к следующей задаче только после проверки под промптом. На шестом шаге сначала прочитай предложенный порядок, затем явно подтверди его; дождись реализации и зелёных проверок перед задачей 7. Все операции с базой, удалением и git выполняй только в учебной копии. Если агент ещё не работает рядом с файлами проекта, сначала открой инструкцию «Claude Code и Codex в одном VS Code»

Что должно произойти | Как проверить | План | Реализация | Приёмка

ИЗ ЧЕГО СОБРАН КАЖДЫЙ ГОТОВЫЙ ПРОМПТ

Контекст задачи → Нужное действие → Безопасные границы → Проверка результата

СЛОВАРЬ НА 30 СЕКУНД

Репозиторий - главная папка проекта с историей изменений. **Diff** - точный список изменённых строк. **Коммит** - сохранение одной законченной версии в истории. **MCP** - как переходник: через него агент получает доступ к внешним инструментам и данным, например к браузеру. Один MCP-сервер может дать несколько инструментов. **CI** - автоматический запуск проверок проекта. **Скилл** - сохранённая инструкция, которую можно запускать повторно, как готовый рецепт

ЗАДАЧА 1 ИЗ 15

Описать требования как наблюдаемый результат

Когда использовать: до первого файла с кодом, когда идея пока звучит одной фразой

Скопировать

```
Мы делаем локальное веб-приложение «Список дел». Пользователь должен создавать задачу, менять её текст, отмечать выполненной, возвращать в работу и удалять после отдельного подтверждения. Данные хранятся в SQLite. Авторизации, совместной работы и уведомлений в первой версии нет.

До написания кода создай документ требований. Для каждой функции напиши: что должно произойти и как это проверить. Добавь границы первой версии, состояния ошибки, правила для пустого названия и критерии приёмки. Не придумывай требования молча: спорные решения вынеси в раздел «Допущения»
```

Ожидаемый результат: документ с функциями, границами и проверяемыми критериями приёмки

Как проверить: у каждого требования есть наблюдаемое действие и однозначный признак успеха

ЗАДАЧА 2 ИЗ 15

Записать правила проекта в действующий файл

Когда использовать: когда одни и те же ограничения приходится повторять в каждом чате

Скопировать

```
Проверь, есть ли AGENTS.md в корне учебной копии проекта. Если есть, используй его как файл правил и покажи, какие правила уже действуют. Если файла нет, предложи создать один AGENTS.md в корне, сначала покажи его полный текст и только после моего подтверждения создай файл. Не создавай CLAUDE.md или второй файл с теми же правилами. Если другой файл уже имеет приоритет, остановись и опиши конфликт без изменений.

Подготовь правила для AGENTS.md: сохранять пользовательские изменения, сначала читать требования, считать задачу выполненной только после проверки, не включать секреты в код, операции с базой и удалением показывать только на тестовой копии, а перед git-коммитом показывать точный diff. Сначала покажи предлагаемую правку или полный текст нового файла. После моего подтверждения внеси её и покажи полный diff файла
```

Ожидаемый результат: один источник правил без правил-дублей

Как проверить: AGENTS.md есть в корне, а полный diff не содержит удаления прежних правил

ЗАДАЧА 3 ИЗ 15

Сначала составить план, не меняя файлы

Когда использовать: когда затронуто несколько частей проекта или цена ошибки выше пары минут

Скопировать

```
Работай только в режиме планирования и пока не меняй файлы. Изучи структуру проекта «Список дел», действующие правила и документ требований. Проверь, ведётся ли здесь история Git. Если да, сохрани исходный вывод `git status --short`, чтобы отличить прежние изменения от новых. Если нет, напиши «Git пока не настроен» и не запускай команды Git.

Предложи три варианта реализации: минимальный, обычный и расширенный. Для каждого укажи затрагиваемые файлы, проверки, риск и объём. Выбери один вариант и объясни выбор. Финал плана: последовательность шагов, после каждого шага указан способ проверки
```

Ожидаемый результат: выбранный маршрут с файлами, рисками и проверками

Как проверить: план готов, но на этом шаге агент не менял файлы

ЗАДАЧА 4 ИЗ 15

Разрабатывать по спецификации, а не по догадке

Когда использовать: после согласования требований и перед реализацией первой функции

Скопировать

```
Возьми документ требований приложения «Список дел» как контракт. Составь таблицу: критерий приёмки, изменение в коде, автоматическая проверка, ручная проверка.

Пока не меняй код. Для каждого критерия первой версии укажи планируемый файл, способ реализации, автоматическую проверку и ручную проверку. Не добавляй авторизацию,
```

```
уведомления и совместную работу. Если какой-то критерий нельзя связать с проверкой,
пометь пробел и остановись до уточнения
```

Ожидаемый результат: карта реализации без функций, которых не просили

Как проверить: каждый критерий связан с будущим изменением и способом проверки

ЗАДАЧА 5 ИЗ 15

Подготовить бриф интерфейса до вёрстки

Когда использовать: когда экран уже понятен по функциям, но ещё не по состояниям и поведению

Скопировать

```
Подготовь бриф интерфейса для приложения «Список дел». Главный экран должен работать
на телефоне шириной 360 пикселей и на ноутбуке.

Опиши иерархию экрана, тексты кнопок, пустое состояние, загрузку, ошибку сохранения и
успешное действие. Добавь работу с клавиатуры, заметный фокус, достаточный контраст и
подтверждение удаления. Не рисуй декоративные блоки без функции. Заверши списком
визуальных критериев, которые можно проверить в браузере
```

Ожидаемый результат: бриф всех состояний одного экрана

Как проверить: есть мобильный вид, клавиатура, ошибки, пустота и подтверждение удаления

ЗАДАЧА 6 ИЗ 15

Разбить реализацию на маленькие проверяемые шаги

Когда использовать: когда план всё ещё слишком крупный для безопасной работы

Скопировать

```
Разбей реализацию приложения «Список дел» на шаги длительностью до одного логического изменения: схема данных, создание задачи, редактирование, смена статуса, удаление с подтверждением, адаптивный интерфейс, итоговые тесты.

Для каждого шага укажи входные файлы, ожидаемый результат, команду проверки и условие остановки. Не смешивай рефакторинг и новую функцию в одном шаге. Сначала покажи весь порядок. После моего явного подтверждения выполняй строго по одному шагу только в учебной копии. Для хранения данных используй лишь тестовую базу `./tmp/tasks-test.sqlite`; её схему и тестовые записи настроим отдельным шагом. Останавливайся на первой красной проверке. В конце покажи, как запустить работающую учебную версию, прежде чем переходить к проверке браузера
```

Ожидаемый результат: серия коротких изменений с отдельной приёмкой

Как проверить: после каждого шага проект работает, а на первой ошибке агент прекращает изменения

ЗАДАЧА 7 ИЗ 15

Подключить только нужный MCP

Когда использовать: когда агенту нужен браузер, внешний сервис или данные, которых нет в папке проекта. Перед подключением можно сверить четыре готовых варианта MCP

Скопировать

```
Для проверки приложения «Список дел» нужен браузер на локальном адресе, то есть по адресу, который открывается только на этом компьютере. Сначала перечисли уже доступные инструменты и проверь, есть ли рабочее браузерное подключение через MCP.

Если оно есть, используй его и выполни безопасный тест: создай задачу «Купить молоко» и проверь, что она появилась в списке. Если подключения нет, предложи один минимальный вариант, дай ссылку на официальную документацию, перечисли разрешения и остановись. До подключения покажи название сервиса и список доступов. Ничего не подключай без моего отдельного подтверждения. Не проси присылать секреты в чат
```

Ожидаемый результат: браузерный тест при готовом MCP или точный план подключения без выдуманного успеха

Как проверить: результат явно помечен: тест выполнен или настройка ожидает действия пользователя

БЕЗОПАСНАЯ ЗОНА ПЕРЕД ЗАДАЧАМИ 8-14

Копия проекта → Тестовая база → Просмотр точного diff → Без push и перезаписи истории

ЗАДАЧА 8 ИЗ 15

Настроить базу только на тестовых данных

Когда использовать: когда функция сохраняет, меняет или удаляет записи

Скопировать

```
Работай только с тестовой базой `./tmp/tasks-test.sqlite`, которую указали в плане реализации. Если файла ещё нет, создай его только в учебной копии проекта. Не открывай и не меняй рабочую базу, даже если найдёшь её в проекте.

Примени существующую миграцию к тестовой базе. Если миграции ещё нет, создай её со схемой id, title, status, created_at и updated_at. Добавь пять тестовых записей и проверки создания, чтения, изменения статуса и удаления. Пустой title должен отклоняться. Покажи абсолютный путь тестовой базы и команду проверки. Не удаляй файл: сначала покажи точный путь и дождись отдельного подтверждения именно этого пути
```

Ожидаемый результат: воспроизводимая схема и тестовый набор данных

Как проверить: CRUD-тесты зелёные, а рабочие файлы базы не менялись

Осторожно: сначала сверяй абсолютный путь. Удалять можно только созданный в этой задаче тестовый файл после отдельного подтверждения пути

ЗАДАЧА 9 ИЗ 15

Проверить риски безопасности без ложной гарантии

Когда использовать: после работающей функции и ещё раз после исправления найденных проблем. До обзора проверь, что **ключи и токены не попали в проект**

Скопировать

```
Проведи обзор безопасности учебного приложения «Список дел». Проверь обработку пользовательского текста, SQL-запросы, вывод данных в HTML, секреты в репозитории, зависимости и доступ к файлам.

Для каждой находки укажи серьёзность, точный файл и строку, возможное последствие и минимальное исправление. Значения секретов не печатай: показывай только тип, имя и путь с маскировкой. Доказательство выбирай безопасное для типа находки: тест, вывод сканера или официальный бюллетень. Не запускай разрушительный сценарий. Отдели подтверждённые проблемы от предположений. После исправлений повтори проверки. В выводе укажи ограничения обзора и не утверждай, что он доказывает полную безопасность
```

Ожидаемый результат: список находок с безопасными доказательствами и ограничениями

Как проверить: для каждой подтверждённой проблемы есть подходящая улика до исправления и чистая повторная проверка после

Безопасная граница: зелёный обзор снижает известные риски, но не доказывает отсутствие всех уязвимостей

ЗАДАЧА 10 ИЗ 15

Найти причину ошибки до исправления

Когда использовать: если появилась повторяемая ошибка. Пример ниже условный: считай ошибку подтверждённой только если она появилась у тебя на экране

Скопировать

```
Проверь условный сценарий: в мобильном браузере дважды нажми «Выполнено». Если появилась ошибка 500, запиши её фактический текст, время и точные шаги. Если ошибки нет, не называй её наблюдаемой и не придумывай журнал. Не исправляй код, пока проблема не воспроизведена.

Попробуй воспроизвести ошибку пять раз. Если она не повторилась, ничего не исправляй: верни журнал попыток и список недостающих данных. Если повторилась, прочитай
```

```
фактический журнал ошибки, найди путь данных от кнопки до записи в тестовой базе и сформулируй одну проверяемую гипотезу. Создай регрессионный тест, добейся красного результата, внеси одно исправление и повтори весь набор тестов
```

Ожидаемый результат: причина и точечное исправление либо доказательный отчёт о невоспроизводимости

Как проверить: есть пять попыток, а при исправлении новый тест красный до правки и зелёный после

ЗАДАЧА 11 ИЗ 15

Проверить пользовательские сценарии целиком

Когда использовать: когда отдельные функции уже работают, но важно проверить путь человека от начала до результата

Скопировать

```
Запусти приложение с временной тестовой базой и проверь в браузере четыре сценария:
1. Пустой список → создание задачи «Купить молоко» → задача видна после перезагрузки
2. Редактирование текста на «Купить овсяное молоко» → новый текст сохраняется
3. Отметка выполненной → возврат в работу
4. Удаление → отмена в подтверждении → повторное удаление → задача исчезла

Проверь ширину 360 и 1280 пикселей и управление клавиатурой. В тестовой среде подмени один ответ сохранения на 500 и проверь, что интерфейс показывает понятную ошибку без потери введённого текста. Для каждого сценария сохрани статус «прошёл» или «не прошёл» и точку отказа
```

Ожидаемый результат: отчёт о четырёх путях и безопасной проверке состояния ошибки

Как проверить: отчёт содержит действие, ожидаемое поведение, фактический результат и статус каждого сценария

ЗАДАЧА 12 ИЗ 15

Убрать лишний код без слепого удаления

Когда использовать: после изменения архитектуры или завершения крупной функции

Скопировать

```
Работай в отдельной учебной копии приложения «Список дел». Сначала только составь список кандидатов на удаление: неиспользуемые файлы, импорты, функции и стили.

Для каждого кандидата покажи доказательство: поиск ссылок, результат анализатора, конфигурацию, маршруты, динамические импорты и затрагиваемые тесты. Ничего не удаляй до таблицы кандидатов. Перед удалением сохрани неизменённую учебную копию для сравнения. После моего отдельного подтверждения удали только позиции из согласованного списка. Если история Git есть, покажи `git diff`; если её нет, перечисли точные удалённые строки и файлы, сверив две учебные копии. Затем собери проект и запусти полный набор тестов и браузерные сценарии
```

Ожидаемый результат: небольшой список подтверждённых удалений без потери поведения

Как проверить: у каждого удаления есть доказательство, сборка, тесты и сценарии остаются зелёными

Перед запуском: удаление выполняется только в отдельной учебной копии после просмотра списка кандидатов

ЗАДАЧА 13 ИЗ 15

Разделить изменения на понятные сохранения в истории проекта

Когда использовать: когда изменения проверены и их нужно разделить на читаемую историю

Скопировать

```
Работай только в учебной копии проекта. Сначала проверь, ведётся ли здесь история Git. Если нет, покажи путь учебной папки и предложи настроить историю версий только в ней. До моего отдельного подтверждения ничего не настраивай. Если я подтвержу настройку, покажи полный список файлов и предложи один начальный коммит: прежние изменения нельзя задним числом честно разделить на этапы. Создай его только после отдельного подтверждения состава файлов. Если я не подтвержу настройку, пропусти эту задачу и объясни, что остальные проверки можно выполнить без истории версий. Если история уже была до этой задачи, покажи `git status`, полный `git diff` и список файлов, созданных именно в этой задаче. Не включай чужие или прежние изменения.
```

```
Только если история уже была, раздели новые изменения максимум на три атомарных коммита по фактическим границам: требования и план, реализация списка задач, тесты и документация. Не создавай пустые коммиты и не дроби связанное изменение ради количества. Для каждого предложи точный состав файлов и сообщение на русском. Перед созданием каждого коммита ещё раз покажи staged diff. Не выполняй push и не переписывай историю
```

Ожидаемый результат: если история уже была - согласованные сохранения; если нет - начальное сохранение либо честный пропуск

Как проверить: состав сохранений согласован заранее либо записано решение работать без истории версий

Только на копии: учебный пример не выполняет push, force push, reset или очистку рабочего дерева

ЗАДАЧА 14 ИЗ 15

Настроить автоматические проверки

Когда использовать: когда одна и та же ошибка возвращается из-за пропущенного теста или проверки

Скопировать

```
Изучи, какие автоматические проверки уже есть в учебной копии проекта «Список дел», и проверь, настроена ли история Git. Если истории нет, настрой только местные команды форматирования и тестов; проверку перед сохранением версии и удалённую автоматическую сборку проекта (CI) отложи. Без подтверждения историю не создавай. Если история есть, предложи минимальную схему: форматирование и быстрые тесты перед сохранением версии. Для CI сначала выясни, есть ли удалённый репозиторий и подходящий сервис. Если их нет, подготовь только конфигурацию для будущего запуска и не называй CI работающим. Отправку проекта в сеть (push) не выполняй.

Сначала прочитай правила выбранного инструмента: разные проверки могут останавливать разные действия. Настрой доступные проверки только в копии. Покажи человеку, где лежат файлы настроек и какую кнопку или команду нажать для местной проверки. Запусти её без отправки проекта: на экране должен появиться отчёт с зелёным статусом. Затем намеренно создай безопасную ошибку форматирования и падающий тест в копии, покажи красный отчёт и верни файлы в зелёное состояние. Проверку перед сохранением версии
```

```
считай работающей только после её отдельного запуска. CI считай работающим только
после реального отчёта в удалённом сервисе; без него укажи «конфигурация
подготовлена, запуск не проверен»
```

Ожидаемый результат: проверенные местные команды и честный статус настройки дополнительных проверок

Как проверить: искусственная ошибка обнаружена, после исправления тот же местный путь проходит; удалённая сборка имеет отдельный отчёт или помечена непроверенной

ЗАДАЧА 15 ИЗ 15

Оформить повторяемую работу в скилл

Когда использовать: когда одна процедура уже повторилась несколько раз и её шаги стабилизировались. Перед сборкой посмотри **пять примеров готовых скиллов**

Скопировать

```
В учебной копии проекта оформи повторяемую приёмку функции в скилл `priemka-fichi`.
Сначала найди действующий формат скиллов в этой среде и не изобретай второй формат.

Задай корректные `name` и `description`, чтобы скилл можно было найти по просьбе
«проверь готовность функции». Вход: документ требований, текущие файлы функции и
тесты; список изменений приложи, если он доступен. При наличии Git проверь `git diff`
и историю коммитов, без Git сравни сохранённые учебные копии, если они есть. Пустой
или неполный список изменений не считай доказательством готовности и не выдумывай
отсутствующий diff. Шаги: сопоставить критерии с текущими файлами, запустить тесты,
проверить пользовательский сценарий, отделить подтверждённые дефекты от
предположений, выдать статус каждого гейта. Выход: короткий отчёт с уликами. Добавь
запрет на удаление, работу с рабочей базой и git push. Сначала запусти скилл по имени
для функции «создание задачи», затем повтори проверку обычной фразой «проверь
готовность функции» и отдельно зафиксируй результат каждого запуска
```

Ожидаемый результат: повторяемая инструкция приёмки с безопасными границами

Как проверить: оба запуска отдельно показывают критерии и статусы

Главный итог: готовый запрос задаёт наблюдаемый результат, границы действия и проверку, после которой можно переходить дальше

Что открыть перед нужным этапом

- Если Claude Code и Codex ещё не работают рядом с файлами проекта, начни с инструкции «Claude Code и Codex в одном VS Code»
- Перед задачей 7 сравни четыре готовых подключения в статье «Четыре MCP для Claude Code»
- Перед проверкой репозитория закрой ключи и токены по инструкции «Как спрятать секреты от Claude Code»
- Для задачи 15 посмотри примеры в материале «Пять скиллов Claude Code»
- Если нужен более широкий набор инструментов, открой «Пять инструментов для Claude Code»

Официальные источники

- Claude Code: планирование до изменения файлов
- Claude Code: правила и память проекта
- Claude Code: скиллы
- Claude Code: поведение автоматических проверок
- Claude Code: подключение MCP
- Codex: работа с AGENTS.md и проверяемыми этапами
- Codex: подключение MCP
- Codex: скиллы

Ты получил инструкцию для своей версии. Как применить эти инструменты к своему проекту?

Что можно делать с нейросетями, если не знаешь кода?

Чтобы почувствовать размах, посмотри Концентрат Ивана Сергеева. ИИ-агенты на твоих глазах собирают продукт, контент, сайты и воронки без кода. Это основа под любую цель: свои услуги, партнёрка, чужой бизнес. Три дня бесплатно, VIP с четвёртым днём - 990 ₽

Забрать Концентрат

Уже знаешь, чего хочешь? Тогда в мастер-группу Ивана: система продаж на ИИ без продюсера и технаря, сопровождение и сообщество

Заполнить анкету предзаписи

Набор открывается редко, кто записался заранее, получает лучшую цену и бонусы. Платить сейчас не нужно

Реклама. ИП Сергеев И. С., ИНН 352511695540. erid: 2Vtzqvk2p6q

ЕЩЁ ОДИН ТЕКСТ

Четыре MCP для Claude Code

Открыть статью